

Search Trees Cont'd

1. Balanced BST

2. Red-Black Tree

1.1 Definition of Red-Black Tree

- A RB-Tree is a BST in which each node has a color, and satisfies the following properties:
 - Every node is either red or black
 - The root is black
 - Every leaf(NIL) is black
 - [no-red-edge] if a node is red, then both its children are black
 - [black-height] For every node, all paths from the node to its descendant leaves contain same number of black nodes

Leaf是表示边界条件的哨兵节点(可以链接到根节点), 为了简单起见, 后面将省略它。

1.2 Black Height

把从节点x往下直到叶子的一条简单路径中黑色节点的个数叫做Black-Height, 记为 $bh(x)$ 。

1.3 Height of RB-Trees

- 在红黑树中, 根为x的子树至少有 $2^{bh(x)} - 1$ 个内部节点。
 - 用数归证明
- 定理: n个节点的红黑树的高度为 $O(\log n)$

1.4 Insert node into an RB-Tree

- Step 1: Color z as red and insert as if the RB-Tree were a BST
- Step 2: Fix any violated properties
 - No fix is needed if z has a black parent after insertion

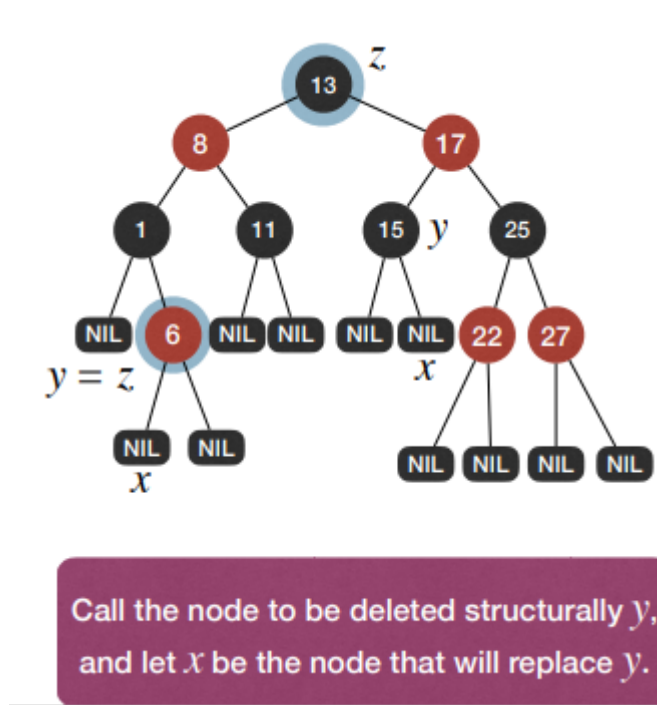
对于Step 2: 调整不符合的情况:

- case 0: z成为了红黑树的根, 把z涂成黑的。
- case 1: z的父节点是红色的, 也就是爷节点是黑色的, 并且它的叔节点是红色的(这时红色节点太多了)。这时需要把z的父节点和叔节点改成黑色的, 爷节点改成红色的。
- case 2: z的父节点是红色的, 叔节点是黑色的。
 - (a): z是它父节点的右孩子。
 - 在z的父节点处进行左旋操作, 然后到case 2(b)。
 - (b): z是它父节点的左孩子。
 - 在z的爷节点处进行右旋操作, 然后再对z的父节点和爷节点重新涂色。

时间复杂度为 $O(\log n)$

1.5 Remove node from an RB-Tree

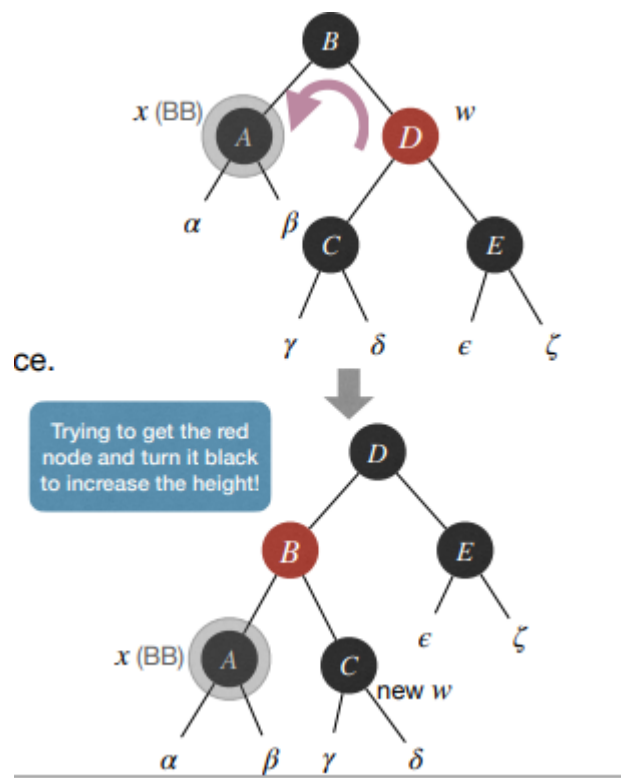
- 如果z的右孩子是一个外部节点（叶子），那么z就是一个要被结构化删除的节点：以z的左孩子为根节点子树将会代替z。
- 如果z的右孩子是一个内部节点，那么让y为以z的右孩子为根节点子树中的最小节点。用y的信息重写z的信息，然后y就成为了要被结构化删除的节点。
- 无论哪种方式，都只需要删除一次结构！
- 应用结构化删除，并且修复冲突的情况。



- Step 1: 识别结构化删除。
- Step 2: 应用结构化删除(维持BST-property)。
- Step 3: 修复违背RB-Tree properties的地方(维持BST-property)。
 - 如果y是一个红色的节点，没有冲突。
 - 如果y是一个黑色的节点并且x是一个红色的节点：把x涂成黑色。
 - 如果y是一个黑色的节点并且x是一个黑色的节点：y对black-height的贡献改变了，因此这与black-height property冲突了，需要修改。

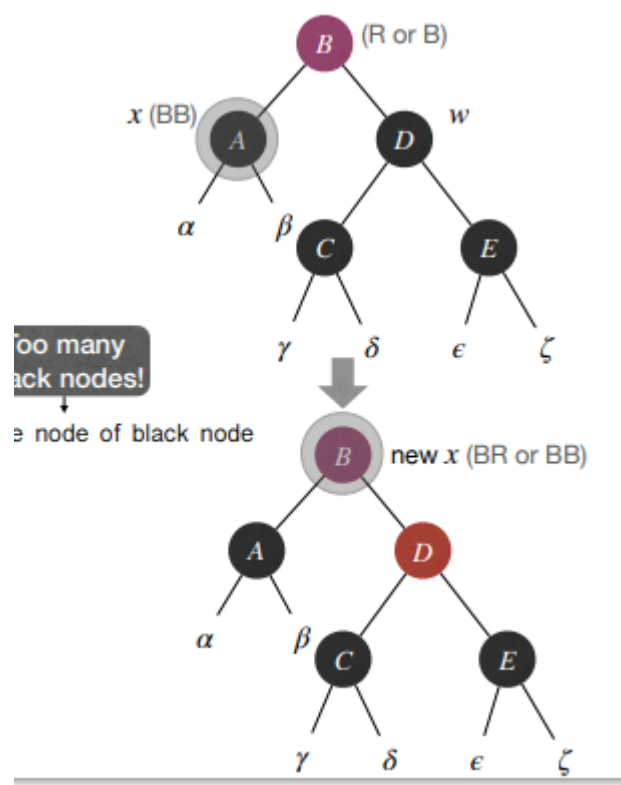
For Step 3, we have four cases:

1. 假设在代替了y的位置后，黑色的x是其父节点的左孩子。并且x的兄弟w是红色的。
- Fix: 旋转并重新涂色。
 - Effect: 把x的兄弟的颜色改成了黑色。



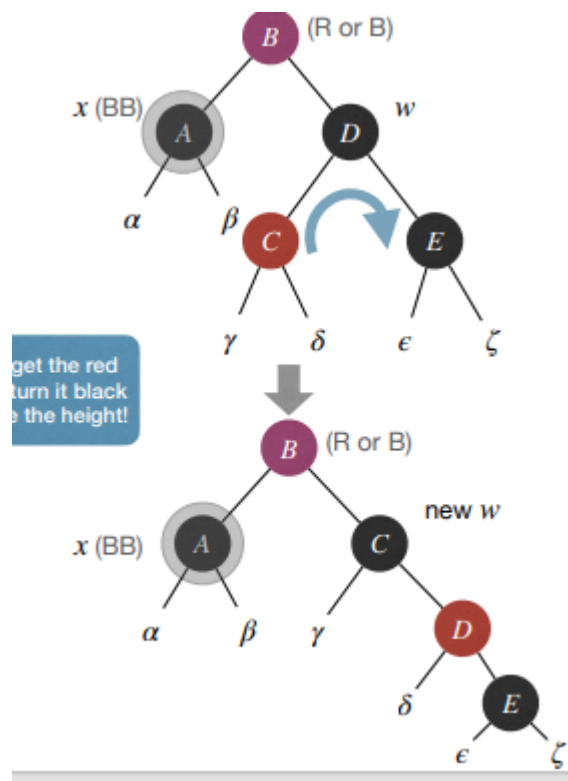
2. 假设 x 是其父节点的左孩子。且 x 的兄弟 w 是黑色的，并且 w 的孩子都是黑色的。

- Fix: recolor and push-up extra blackness in x
- Effect: 要么结束了，要么我们push-up了 x



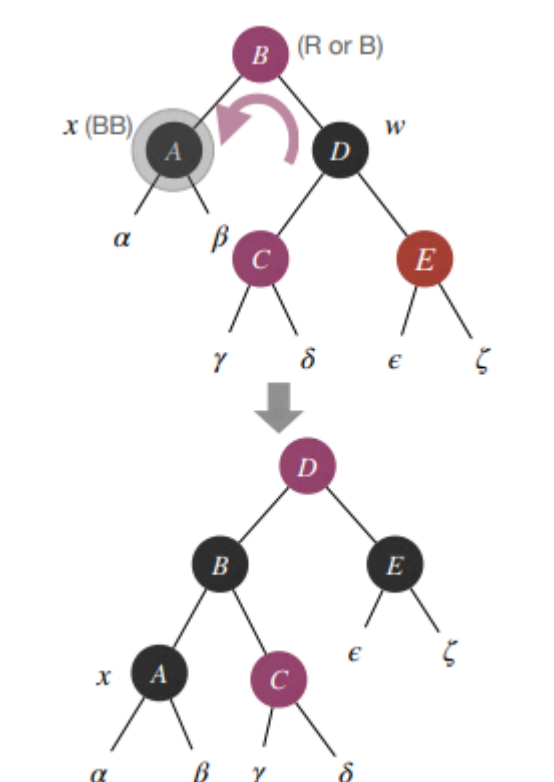
3. 假设 x 是其父节点的左孩子， x 的兄弟 w 是黑色的，并且 w 的左孩子是红色的，右孩子是黑色的。

- Fix: 旋转并且重新涂色
- Effect: w 的右孩子变成红色的



4. 假设 x 是其父节点的左孩子， x 的兄弟 w 是黑色的，且 w 的右孩子是红色的。

- Fix: 旋转并且重新涂色
- Effect: We are done!



时间复杂度为: $O(\log n)$

3.Skip List

3.1 Definition of Skip List

- 跳表是可以实现二分查找的有序链表。
- 建立多层索引，逐层减少元素数量。
- 每层花费 $O(1)$ 时间，一共花费 $O(\log n)$ 时间。
- 空间复杂度为 $O(n)$ 。

3.2 The real Skip List

```
1 Insert(L,x):
2   level:=1,done:=false
3   while !done
4     x:=y
5     Insert x into level k list
6     Flip a fair coin:
7       with probability 1/2 -> done:=true
8       with probability 1/2 -> k:= k+1
```

时间复杂度为 $O(\log n)$

Max level of n-element SkipList is $O(\log n)$ with high probability